为什么要学交叉验证
1 第一部分：定义与方法
2 1. 交叉验证的定义
3 2. 为什么需要 CV：一个可控的模拟
4 3. 主要方法与关键问题
5 3.1 留出法（Validation Set / Hold-out）
6 3.2 K 折交叉验证
7 3.3 留一法（LOOCV）与帽子矩阵捷径
8 3.4 重复 K 折（Repeated K-fold）
9 3.5 数据泄漏（Data Leakage）
10 第二部分：适用情况
11 4. 什么情况下该用哪种 CV
12 4.1 决策树：先问数据是什么结构
13 4.2 嵌套 CV：当"选模型"本身也要被评估
14 4.3 分组 / 聚类数据：留一组法
15 4.4 时间序列：不能用随机 K 折
16 4.5 分层 K 折：不平衡数据
17 4.6 CV 做不到什么
18 第三部分：动手做
19 5. 完整案例：seatpos 上从头实现 CV
20 6. 常见陷阱与最佳实践
21 附录

Cross-Validation (交叉验证)

MTH303 — 从定义到适用场景，附完整 R 实现

MTH303 Teaching Group

2026-10-14

为什么要学交叉验证

> **一句话 / in one sentence**
>
> **交叉验证（cross-validation, CV）是用同一批数据，估计模型在"没见过的数据" 上会有多好的一种方法。**
>
> *Cross-validation estimates how well a model will perform on data it has not seen, using only the data you have.*

0.1 0.1 它解决的是什么问题

回忆你们在**变量选择**那一节学到的三件事。stepwise 的三个毛病里， **第一个毛病就是 CV 要治的病**：

stepwise 的毛病	具体表现	CV 能治吗
① 准则都是样本内的	adjusted R^2、AIC、BIC 都用同一批数据既选模型又评模型	✅ **这就是 CV 存在的理由**
② 搜索是贪心的	一次只动一个变量，进去的出不来	❌ 换准则救不了，要靠全子集或正则化
③ 结果不稳定	起点、方向、并列时的取舍都是任意约定	⚠️ 部分能（CV 自己也不稳，见 §3.4）

核心矛盾：

> 你**手头只有一批数据**，但你真正想知道的是 **"这个模型拿去用在明天的新病人/新客户身上，会错多少？"**
>
> 样本内误差（in-sample error）**永远**回答不了这个问题—— 因为它按定义就是拿同一批数据算的。 **CV 就是那个"假装有新数据"的技巧。**

0.2 0.2 三个必须先分清的"误差"

这是全文最重要的一组定义。**必须把下面三个词严格分开**，否则后面全乱：

名称	符号	定义	能不能直接算出来
训练误差 Training error	$\overline{\mathrm{err}}$	在**拟合用的这批数据**上的平均损失	✅ 直接算（就是 `mean(resid(m)^2)`）
测试误差 Test error	$\mathrm{Err}_{\mathcal{T}}$	在**一批固定的新数据** $\mathcal{T}$ 上的平均损失	❌ 除非你真有一批新数据

名称	符号	定义	能不能直接算出来
期望测试误差 Expected test error	Err	对**所有可能的新数据**取期望	❌ 理论上存在，算不出来

形式化地写： 设损失是平方误差，$\hat{f}$ 是在训练集 $\{(x_i, y_i)\}_{i=1}^{n}$ 上拟合的模型，

$$\underbrace{\overline{\mathrm{err}} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{f}\left(x_i\right)\right)^2}_{\text{训练误差：可以算，但有偏}} \qquad \underbrace{\mathrm{Err}_{\mathcal{T}} = \frac{1}{|\mathcal{T}|}\sum_{i\in\mathcal{T}}\left(y_i - \hat{f}\left(x_i\right)\right)^2}_{\text{测试误差：要一批新数据}}$$

$$\underbrace{\mathrm{Err} = \mathbb{E}\left[\left(Y - \hat{f}\left(X\right)\right)^2\right]}_{\text{期望测试误差：真正的目标}}$$

> 🔴 **为什么训练误差不能用？**
>
> 因为 $\hat{f}$ **是看着这批数据挑出来的。**它当然在这批数据上表现好—— 这不是证据，这是**循环论证。**
>
> 而且越灵活（越复杂）的模型，训练误差只会越小。 用训练误差选模型，你**永远**会选到最复杂的那一个。 （§2 会用一个模拟把这件事量化出来。）

CV 的定位：

> **CV 不是测试误差，也不是期望测试误差。 CV 是 Err 的一个估计量（estimator）。** 它有偏、有方差，需要讨论它估得准不准——这正是 §3 的内容。

---

1 第一部分：定义与方法

2 1. 交叉验证的定义

2.1 1.1 一般定义

设：

- 数据 $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$；
- 损失函数 $L(y, \hat{y})$（回归常用 $L = (y - \hat{y})^2$；分类常用 0-1 损失）；

- $\mathcal{A}$ 是**一个完整的建模流程**（fitting procedure）： 输入训练集，输出一个预测函数 $\hat{f}$。 **注意 $\mathcal{A}$ 可以包含"选变量""调超参数""做标准化"等所有步骤。**

把 $\mathcal{D}$ 随机划分成 K 个大小相近的折（folds）$\mathcal{D}_1, \ldots, \mathcal{D}_K$。 对每个 $k = 1, \ldots, K$：

1. 用**除第 k 折以外**的数据训练：$\hat{f}^{(-k)} = \mathcal{A}\big(\mathcal{D} \setminus \mathcal{D}_k\big)$；
2. 在**留出的第 k 折**上算损失：

$$\mathrm{CV}_k = \frac{1}{|\mathcal{D}_k|} \sum_{i \in \mathcal{D}_k} L\big(y_i,\ \hat{f}^{(-k)}(x_i)\big)$$

K 折交叉验证的估计量就是这 K 个值的平均：

$$\boxed{\widehat{\mathrm{CV}}_K = \frac{1}{K} \sum_{k=1}^{K} \mathrm{CV}_k = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{|\mathcal{D}_k|} \sum_{i \in \mathcal{D}_k} L\big(y_i,\ \hat{f}^{(-k)}(x_i)\big)}$$

三个必须注意的细节：

1. **下标 $(-k)$ 是全部关键。** 预测 x_i 用的模型**没有见过** y_i。 这就是"假装有新数据"的全部含义。
2. **每一折的预测都来自不同的模型。** 你有 K 个不同的 $\hat{f}^{(-k)}$， 最后**并不选其中一个**，只是把它们在留出折上的表现平均起来。
3. **$\mathcal{A}$ 是整个流程。** 如果流程里包含"先选变量再拟合"， 那么**选变量也必须在折内做。**做错了就是数据泄漏（§3.6）。

2.2 1.2 一个最小可运行的例子

用 `seatpos` （38 位驾驶员的体型与座位位置），比较两个模型：

```
sp <- seatpos
nrow(sp)
#> [1] 38
```

```r
# 一个通用的 K 折 CV 函数 ----------------------------------------------
cv_error <- function(formula, data, K = 10, seed = 1) {
  set.seed(seed)
  n    <- nrow(data)
  fold <- sample(rep(1:K, length.out = n))     # 尽量均分到 K 折
  errs <- numeric(K)
  for (k in 1:K) {
    train <- data[fold != k, ]
    test  <- data[fold == k, ]
    fit   <- lm(formula, data = train)          # 只用训练折拟合
    pred  <- predict(fit, newdata = test)       # 在留出折上预测
    errs[k] <- mean((test$hipcenter - pred)^2)
  }
  mean(errs)
}

cv_error(hipcenter ~ Ht, sp, K = 10)
#> [1] 1440.33
cv_error(hipcenter ~ Age + HtShoes + Leg, sp, K = 10)
#> [1] 1348.37
```

短短 12 行就是 CV 的全部。 后面所有花哨的变体，都是在"**怎么分折**"和"**折内做什么**"这两个问题上做文章。

2.3 1.3 CV 估计的到底是什么

这一点很多人搞错，值得单独强调。

CV 估计的不是"某一个模型的测试误差"，而是"你这套建模流程 $\mathcal{A}$ 的测试误差"。

区别在哪？

- 如果你固定一个模型（比如 `hipcenter ~ Ht`），那 $\mathcal{A}$ 就是"拟合这一个模型"，CV 估的就是它的测试误差；
- 但如果 $\mathcal{A}$ 是"**先用 CV 选出最好的模型，再拟合它**"，那么 CV 估的是**这个选模型流程**的测试误差——**这就是嵌套 CV（§4.2）要处理的事。**

> 💡 **记住这句话：CV 衡量的是流程，不是模型。** 你往流程里塞进越多"看着数据做决定"的步骤（选变量、挑超参数、挑阈值……），用普通 CV 报出来的数字就越乐观。

2.4 1.4 损失函数的选择

CV 的框架不依赖具体损失，换损失就是换"什么叫错"：

任务	常用损失	R 里怎么写
回归	平方误差 $(y-\hat{y})^2$ → MSE	`mean((y - pred)^2)`
回归（抗异常值）	绝对误差 $\lvert y-\hat{y}\rvert$ → MAE	`mean(abs(y - pred))`
分类	0-1 损失 → 错误率	`mean(y != pred)`
分类（概率）	对数损失 / 偏差	`-mean(y*log(p) + (1-y)*log(1-p))`
计数	Poisson 偏差	`-2*mean(dpois(y, pred, log=TRUE))`

选哪个损失，取决于你关心什么。 如果异常值很贵，用 MAE； 如果大误差特别不可接受，用 MSE（它惩罚大误差更重）。

---

3 2. 为什么需要 CV：一个可控的模拟

光讲道理不够，我们造一个"知道标准答案"的数据集，把过拟合量出来。

3.1 2.1 设定

```
set.seed(303)
n <- 100; p <- 12
X <- matrix(rnorm(n * p), n, p)
colnames(X) <- paste0("x", 1:p)

# 真模型：只有前 4 个变量有用，其余 8 个系数严格为 0
beta <- c(1.5, -1.2, 0.8, 0.6, rep(0, p - 4))
y    <- drop(X %*% beta) + rnorm(n, sd = 2)     # 噪声 sd = 2

dat <- data.frame(y, X)
c(真实_R2 = 1 - 4 / (sum(beta^2) + 4))
#>  真实_R2
#> 0.539701
```

所以我们知道：真模型是 `x1 + x2 + x3 + x4`，其他 8 个变量纯属噪声。

3.2 2.2 四个准则，一条模型序列

按变量个数从 1 到 12 依次加变量，对每个模型算四个数：

- **训练误差**（样本内 MSE）
- **LOOCV**（留一法，用帽子矩阵公式，§3.3）
- **10 折 CV**

- **Adjusted R^2**

```
# 10 折 CV（可复用）
kfold_mse <- function(formula, data, K = 10, seed = 303) {
  set.seed(seed)
  n    <- nrow(data)
  fold <- sample(rep(1:K, length.out = n))
  mean(sapply(1:K, function(k) {
    tr <- data[fold != k, ]; te <- data[fold == k, ]
    mean((te$y - predict(lm(formula, data = tr), newdata = te))^2)
  }))
}

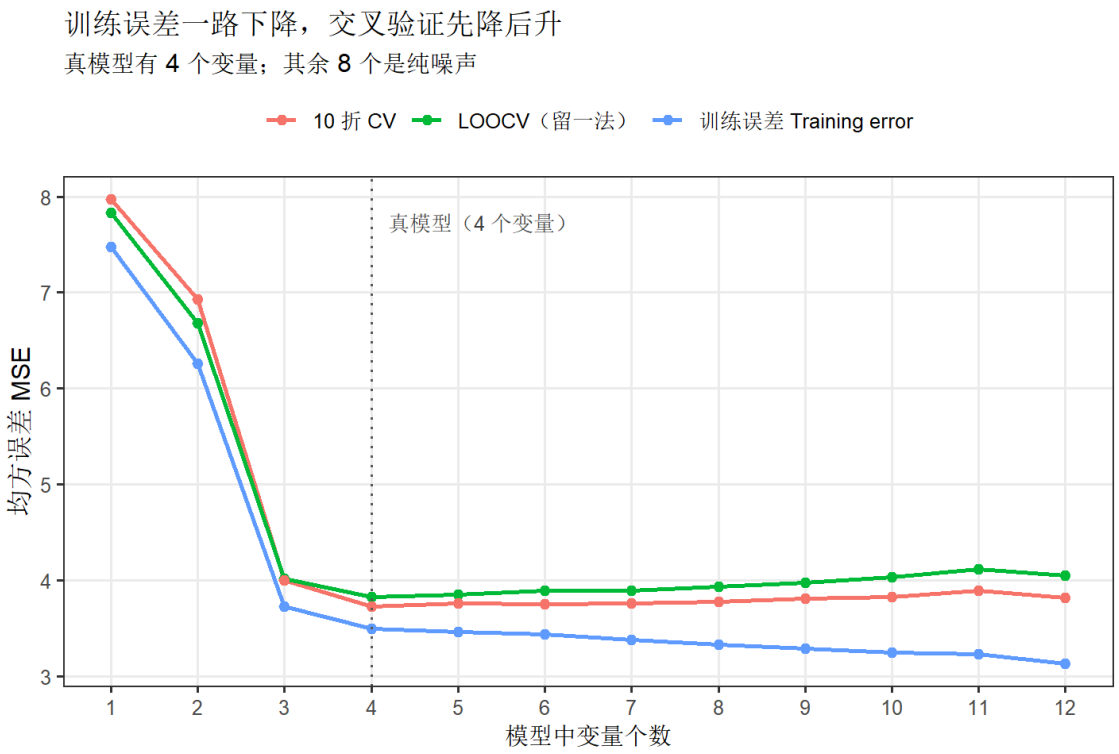
res <- lapply(1:12, function(s) {
  f <- reformulate(colnames(X)[1:s], response = "y")
  m <- lm(f, data = dat)
  e <- resid(m); h <- hatvalues(m)
  data.frame(
    vars        = s,
    k_params    = s + 1,
    train_MSE   = mean(e^2),
    LOOCV       = mean((e / (1 - h))^2),          # 帽子矩阵公式，见 §3.3
    kfold10     = kfold_mse(f, dat, K = 10),
    adj_R2      = summary(m)$adj.r.squared
  )
}) |> do.call(what = rbind)

res |> mutate(across(where(is.numeric), \(v) round(v, 4)))
#>    vars k_params train_MSE  LOOCV kfold10 adj_R2
#> 1     1        2    7.4717 7.8277  7.9682 0.2501
#> 2     2        3    6.2545 6.6806  6.9262 0.3658
#> 3     3        4    3.7254 4.0129  3.9955 0.6183
#> 4     4        5    3.4922 3.8262  3.7251 0.6384
#> 5     5        6    3.4601 3.8513  3.7613 0.6380
#> 6     6        7    3.4396 3.8935  3.7523 0.6362
#> 7     7        8    3.3782 3.8938  3.7592 0.6388
#> 8     8        9    3.3306 3.9330  3.7784 0.6400
#> 9     9       10    3.2896 3.9756  3.8088 0.6405
#> 10   10       11    3.2436 4.0292  3.8255 0.6415
#> 11   11       12    3.2305 4.1146  3.8906 0.6389
#> 12   12       13    3.1305 4.0515  3.8158 0.6461
```

```
res_long <- res |>
  select(vars, train_MSE, LOOCV, kfold10) |>
  pivot_longer(-vars, names_to = "criterion", values_to = "MSE") |>
  mutate(criterion = recode(criterion,
    train_MSE = "训练误差 Training error",
    LOOCV     = "LOOCV（留一法）",
    kfold10   = "10 折 CV"
  ))

best <- res$vars[which.min(res$LOOCV)]

ggplot(res_long, aes(vars, MSE, colour = criterion)) +
  geom_line(linewidth = 0.9) + geom_point(size = 2) +
  geom_vline(xintercept = 4, linetype = 3, colour = "grey35") +
  annotate("text", x = 4.2, y = max(res_long$MSE) * 0.97,
           label = "真模型（4 个变量）", hjust = 0, size = 3.4, colour = "grey25") +
  scale_x_continuous(breaks = 1:12) +
  labs(x = "模型中变量个数", y = "均方误差 MSE", colour = NULL,
       title = "训练误差一路下降，交叉验证先降后升",
       subtitle = "真模型有 4 个变量；其余 8 个是纯噪声") +
  theme_bw(base_size = 12) +
  theme(legend.position = "top", panel.grid.minor = element_blank())
```



这张图是全文的核心。 逐条读：

准则	最小值出现在	说明
训练误差	12 个变量（全模型）	**一路下降，永远选最复杂的** ❌

准则	最小值出现在	说明
LOOCV	**4 个变量**	✅ 命中真模型附近
10 折 CV	**4 个变量**	✅ 命中真模型附近

关键数字对比（全模型 vs 真模型）：

```
res |>
  filter(vars %in% c(4, 12)) |>
  select(vars, train_MSE, LOOCV, kfold10) |>
  mutate(across(where(is.numeric), \(v) round(v, 4)))
#>   vars train_MSE  LOOCV kfold10
#> 1    4    3.4922 3.8262  3.7251
#> 2   12    3.1305 4.0515  3.8158
```

> 💡 **从这张表读出的三件事：**
>
> **① 训练误差是"训练集上的记忆能力"，不是"预测能力"。** 全模型的训练误差 3.13 比真模型 3.492 **还低**——但它多了 8 个没用的变量。 多出来的那 8 个变量把噪声也记住了。
>
> **② CV 能在没有新数据的情况下，看出过拟合。** 全模型的 CV 误差 4.051 比真模型 3.826 **高 6%。 CV 反过来了——它惩罚复杂度。**
>
> **③ 但 CV 也不是完美的：** 它在 4 附近触底，但 **最低点不一定正好在 4**（本例 LOOCV 选 4、10 折选 4）。相邻的几个 `MSE` 差距很小， 属于"**在统计噪声范围内**"。**不要把 CV 的最小值点当成精确答案。**

---

4 3. 主要方法与关键问题

5 3.1 留出法（Validation Set / Hold-out）

最原始的做法： 把数据分成两部分，一部分训练、一部分评估。

优点： 简单、快、**独立于模型**（任何模型都能用）。

缺点（两个，都很致命）：

1. **只用了一部分数据训练** —— 剩下的数据"浪费"了。样本量本来就小时，这很痛。
2. **结果高度依赖"怎么分"** —— 换一个随机种子，误差估计可能差很多。

```r
# 同一个模型，只换划分方式，看估计值抖动多大
holdout_mse <- function(data, prop = 0.7, seed) {
  set.seed(seed)
  n     <- nrow(data)
  idx   <- sample(n, size = floor(prop * n))
  train <- data[idx, ]; test <- data[-idx, ]
  m     <- lm(y ~ x1 + x2 + x3 + x4, data = train)
  mean((test$y - predict(m, newdata = test))^2)
}

holdout_est <- sapply(1:200, function(b) holdout_mse(dat, 0.7, seed = b))
c(mean = mean(holdout_est), sd = sd(holdout_est),
  min = min(holdout_est), max = max(holdout_est))
#>     mean       sd      min      max
#> 3.876761 0.942259 1.992632 6.705958
```

> 🔴 **单次留出法的估计值可以抖到 ±0.9 以上。** 用它来选模型，等于把结论交给随机数。 **这就是为什么需要 K 折：用"平均 K 次"来压住这个方差。**

---

6 3.2 K 折交叉验证

做法： 分成 K 折，轮流留一折出来当测试集，共拟合约 K 次。

K 怎么选？ 这是本节的中心问题。K 同时影响**偏差**和**方差**：

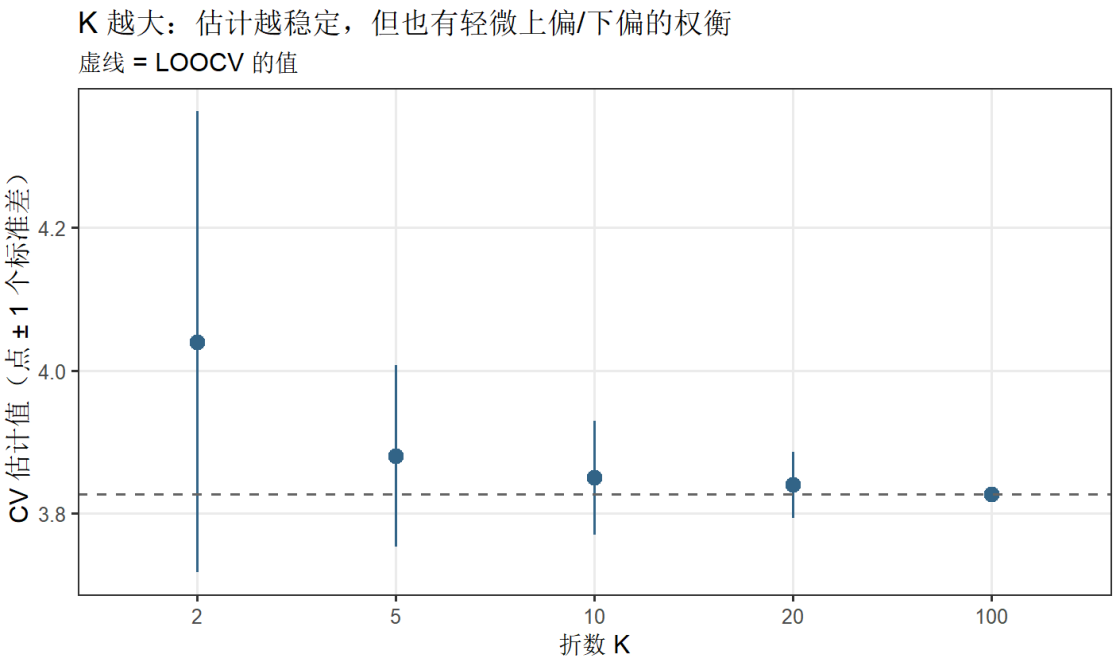
	训练集大小	偏差	方差	计算量
K 小（如 2、5）	小	**偏大**（低估真实表现）	大	小
$K = 10$	$0.9n$	略偏大	中	中
$K = n$（LOOCV）	$n - 1$	**几乎无偏**	**大**	大（但有捷径）

直观理解这个权衡：

- **偏差从哪来：** 用 $n(1 - 1/K)$ 个点训练，比用 n 个点训练**学得差一点**， 所以 CV 误差会**略微高估**真实误差。K 越大，训练集越接近 n，偏差越小。
- **方差从哪来：** K 折之间**高度相关**（训练集几乎重叠）。 K 越大，K 个估计量之间的相关性越强，**平均掉的东西越少**， 而且每个留出折越小、目标量越"抖"。所以 $K = n$ 时方差最大。

```r
# 用同一个模型，看 K 如何影响估计量与它的稳定性
B <- 200
k_tab <- do.call(rbind, lapply(c(2, 5, 10, 20, n), function(K) {
  v <- sapply(1:B, function(b) kfold_mse(y ~ x1 + x2 + x3 + x4, dat, K = K, seed
= b))
  data.frame(K = K, mean = mean(v), sd = sd(v))
}))
k_tab |> mutate(across(c(mean, sd), \(v) round(v, 4)))
#>     K   mean     sd
#> 1   2 4.0400 0.3230
#> 2   5 3.8802 0.1269
#> 3  10 3.8498 0.0798
#> 4  20 3.8397 0.0463
#> 5 100 3.8262 0.0000
```

```r
ggplot(k_tab, aes(factor(K), mean)) +
  geom_pointrange(aes(ymin = mean - sd, ymax = mean + sd),
                  colour = "steelblue4", linewidth = 0.7) +
  geom_hline(yintercept = res$LOOCV[4], linetype = 2, colour = "grey40") +
  labs(x = "折数 K", y = "CV 估计值（点 ± 1 个标准差）",
       title = "K 越大：估计越稳定，但也有轻微上偏/下偏的权衡",
       subtitle = "虚线 = LOOCV 的值") +
  theme_bw(base_size = 12) + theme(panel.grid.minor = element_blank())
```



实测结果说明了什么：

- $K = 2$：均值 **4.04**，标准差 **0.323** —— **偏差最大、抖动也最大**，两头不讨好；
- $K = 10$ 到 20：均值稳定在 **3.85** 附近， 标准差降到 **0.08**；

- $K = n$（LOOCV）：均值 **3.826**，标准差 **0** —— **均值最低（偏差最小），但注意它的方差不是最小。**

> ✅ **实践建议：$K = 5$ 或 $K = 10$。** 这是几十年经验得出的甜点：偏差已经很小，计算量可接受，方差也可控。 $K = 10$ **是默认选择**，$K = 5$ 在样本量大、算得慢时用。
>
> $K = n$**（LOOCV）什么时候用？** 样本量很小（$n < 50$）时， 或者当你有 §3.3 那种闭式捷径、算起来几乎免费时。

---

7 3.3 留一法（LOOCV）与帽子矩阵捷径

LOOCV = $K = n$ **的 K 折**：每次只留 1 个观测，用其余 $n - 1$ 个拟合，预测那 1 个。

$$\text{LOOCV} = \frac{1}{n} \sum_{i=1}^{n} L\big(y_i,\ \hat{f}^{(-i)}(x_i)\big)$$

问题： 要拟合 n 次。$n = 10^5$ 时不可行。

但对线性模型（最小二乘）有一个漂亮的闭式解：

$$\boxed{\text{LOOCV} = \frac{1}{n} \sum_{i=1}^{n} \left(\frac{e_i}{1 - h_{ii}} \right)^2}$$

其中 e_i 是普通残差，h_{ii} 是**帽子矩阵的对角元（杠杆值）。**

这个公式为什么成立（直觉）：

- h_{ii} 大 ⇒ 第 i 个点在 x 空间很"特别" ⇒ 回归线**被它拉过去**了；
- 于是 e_i 会**偏小**——模型"迁就"了这个点；
- $1/(1 - h_{ii})$ 就是**把这个迁就修回来的修正量。** $h_{ii} \to 1$ 时修正量爆炸，正说明这种点的留一残差极不可靠。

跟你们学过的帽子矩阵讲义直接接上： 这就是杠杆值的用途之一。

```r
# 两条路都算一遍，验证恒等式
loocv_hat <- function(formula, data) {
  m <- lm(formula, data)
  mean((resid(m) / (1 - hatvalues(m)))^2)
}
loocv_refit <- function(formula, data) {          # 老老实实重拟合 n 次
  n <- nrow(data)
  mean(sapply(1:n, function(i) {
    m <- lm(formula, data = data[-i, ])
    (data[[all.vars(formula)[1]]][i] - predict(m, newdata = data[i, ]))^2
  }))
}

f_test <- y ~ x1 + x2 + x3 + x4
c(hat_matrix_shortcut = loocv_hat(f_test, dat),
  brute_force_refit   = loocv_refit(f_test, dat),
  difference          = loocv_hat(f_test, dat) - loocv_refit(f_test, dat))
#> hat_matrix_shortcut   brute_force_refit          difference
#>         3.82621e+00         3.82621e+00        -1.33227e-15
```

两者差异在 1.33227e-15 量级—— 就是浮点误差。 恒等式成立。

> 💡 **教学要点：** 这个捷径**只对线性模型且损失是平方误差时成立。** 逻辑回归、随机森林、任何带超参数调优的流程——**都不能用**， 老老实实重拟合。 而且很多包的"LOOCV"默认就用这个公式，**你至少要能判断它对你的模型成不成立。**

---

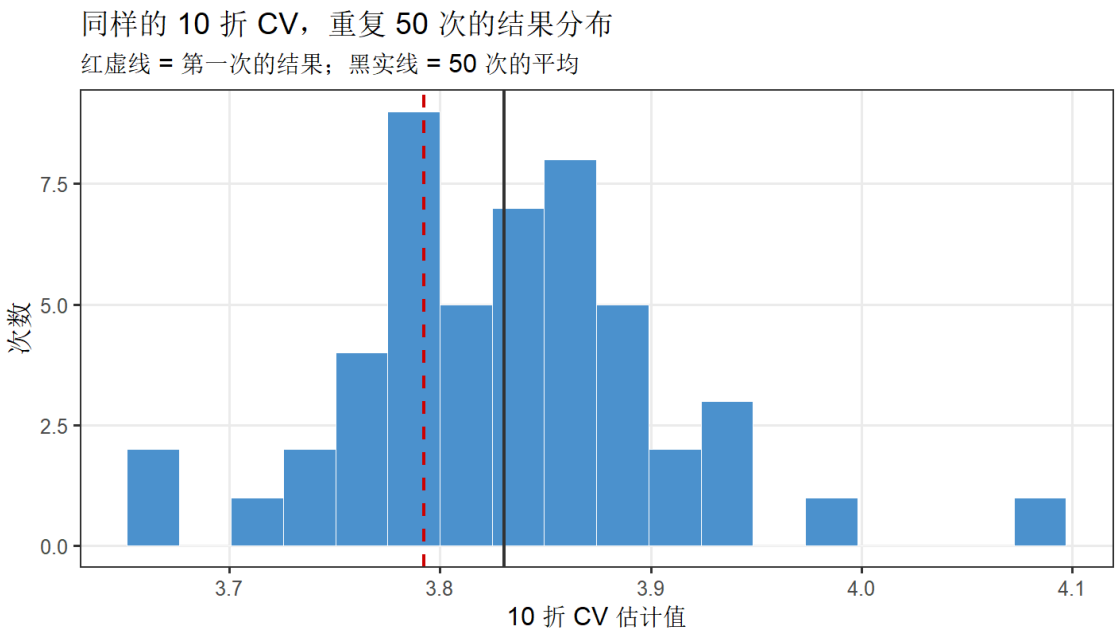
8 3.4 重复 K 折（Repeated K-fold）

CV 估计本身**有随机性**：换个随机种子，结果就变。 **重复 K 折**就是把整个 K 折过程**独立重复 R 次**（每次重新划分），再平均。

- **好处**：方差显著降低；还能**顺便给出一个标准差**，让你知道这个估计有多稳。
- **代价**：计算量 × R。
- **注意**：重复 R 次并**不能降低"训练集比全体小"带来的偏差**—— 它只压方差，不纠偏差。

```r
# 单个 10 折 vs 重复 10 折
one    <- kfold_mse(y ~ x1 + x2 + x3 + x4, dat, K = 10, seed = 1)
reps   <- sapply(1:50, function(r) kfold_mse(y ~ x1 + x2 + x3 + x4, dat, K = 10,
seed = 1000 + r))
c(single_10fold = one,
  mean_of_50    = mean(reps),
  sd_of_50      = sd(reps),
  range_lo      = min(reps),
  range_hi      = max(reps))
#> single_10fold    mean_of_50      sd_of_50      range_lo      range_hi
#>     3.7924137     3.8305600     0.0754582     3.6636607     4.0847469
```

```r
ggplot(data.frame(x = reps), aes(x)) +
  geom_histogram(bins = 18, fill = "steelblue3", colour = "white", linewidth = 0.
3) +
  geom_vline(xintercept = one, linetype = 2, colour = "red3", linewidth = 0.8) +
  geom_vline(xintercept = mean(reps), colour = "grey20", linewidth = 0.8) +
  labs(x = "10 折 CV 估计值", y = "次数",
       title = "同样的 10 折 CV，重复 50 次的结果分布",
       subtitle = "红虚线 = 第一次的结果；黑实线 = 50 次的平均") +
  theme_bw(base_size = 12) + theme(panel.grid.minor = element_blank())
```



> 🔴 **这张图值得盯着看几秒。** 第一次的结果（红虚线）落在分布的左侧， 与平均差了 0.038。
>
> **如果两个候选模型的 CV 值只差 0.05，而 CV 自身的标准差是 0.075——那么"谁更好"这个结论基本是随机的。**
>
> **所以：报告 CV 结果时，一定要报告不确定性。** 单次 CV 的一个数字，不足以支撑"模型 A 优于模型 B"的结论。 最省事的做法就是**重复 K 折，报告均值 ± 标准差。**
>
> **另一个更好用的技巧：**"**一倍标准差规则**"（one-standard-error rule）—— 在所有 CV 值落在"最优值 + 1 个标准差"范围内的模型里，**选最简单的那个。** 这是对"CV 最小值有噪声"的直接补救。

---

9 3.5 数据泄漏（Data Leakage）

这是新手最常犯、后果最严重的错误。

> **规则：任何"看着数据做的决定"，都必须放进折内。** *Anything that looks at the data must happen inside the fold.*

什么叫"看着数据做决定"？

步骤	泄漏的做法 ❌	正确的做法 ✅
标准化 / 中心化	用**全部**数据算均值和标准差	**折内**算，用训练折的均值去中心化测试折
缺失值填补	用**全部**数据算中位数填补	**折内**算中位数
特征筛选	用**全部**数据算相关系数挑变量	**折内**挑变量
主成分分析	用**全部**数据做 PCA	**折内**做 PCA，把载荷应用到测试折
选超参数	用**全部**数据调参	**折内**调参（→ 嵌套 CV，§4.2）
去掉离群值	用**全部**数据识别离群点	**折内**识别

跑一个极端例子。 造 $n = 60$ 个观测、$p = 5000$ 个**纯噪声**变量， y 和它们**完全独立**。所以**真实的测试误差就是 1**（$y \sim N(0,1)$）。

我们用"先在全样本上挑相关系数最高的 20 个变量，再拟合"这个流程：

```r
set.seed(7)
n2 <- 60; p2 <- 5000
X2 <- matrix(rnorm(n2 * p2), n2, p2)
colnames(X2) <- paste0("v", 1:p2)
y2 <- rnorm(n2)                       # y 与所有 X 完全独立
dd <- data.frame(y = y2, X2)

# 挑出与 y 相关系数绝对值最大的 20 个变量名
top20 <- function(yv, Xm) {
  cors <- sapply(seq_len(ncol(Xm)), function(j) cor(yv, Xm[, j]))
  colnames(Xm)[order(abs(cors), decreasing = TRUE)][1:20]
}
```

```r
# ❌ 泄漏：选变量时看到了全部数据
leaky <- function(K = 10, seed = 1) {
  set.seed(seed)
  sel  <- top20(dd$y, X2)                 # <-- 看了全部 60 个观测
  f    <- reformulate(sel, "y")
  fold <- sample(rep(1:K, length.out = n2))
  mean(sapply(1:K, function(k) {
    tr <- dd[fold != k, ]; te <- dd[fold == k, ]
    mean((te$y - predict(lm(f, data = tr), newdata = te))^2)
  }))
}

# ✅ 正确：选变量只在训练折内做
honest <- function(K = 10, seed = 1) {
  set.seed(seed)
  fold <- sample(rep(1:K, length.out = n2))
  mean(sapply(1:K, function(k) {
    idx <- fold != k
    sel <- top20(dd$y[idx], X2[idx, , drop = FALSE])   # <-- 只看训练行
    f   <- reformulate(sel, "y")
    mean((dd$y[!idx] - predict(lm(f, data = dd[idx, ]),
                               newdata = dd[!idx, ]))^2)
  }))
}

B <- 30
leak_v   <- sapply(1:B, function(b) leaky(seed = b))
honest_v <- sapply(1:B, function(b) honest(seed = b))

c(leaky_outside_fold = mean(leak_v),
  honest_inside_fold = mean(honest_v),
  truth_y_is_noise   = 1)
#> leaky_outside_fold honest_inside_fold   truth_y_is_noise
#>           0.334156           1.605836           1.000000
```

> 💡 **结论非常刺眼：**
>
> - **泄漏的做法报出 0.334** —— 看起来"还不错"， 甚至比真值 1 还小；
> - **正确的做法报出 1.606** —— 诚实地反映了 "这些变量根本没用，误差大约是 1 加上选变量的额外波动"；
> - **真实值是 1**（y 是纯噪声）。
>
> 泄漏把误差**低估了 79%。** 而在这个例子里，y **和任何变量都没有任何关系**—— 泄漏的做法却让你以为找到了规律。
>
> **这就是为什么数据科学竞赛里"排行榜分数"经常在私榜上崩掉。**

> ⚠️ **一个诚实的补充：** 并不是所有预处理泄漏都会造成明显偏差。 例如**只做中心化/标准化**、模型是**普通最小二乘**时， 线性变换不影响 $\hat{\beta}$ 和预测， 这时"先标准化再 CV"和"折内标准化"**结果完全一样。** 我实测过，差异是 0.0000。
>
> **但这不能成为习惯。** 一旦换成需要调超参数的模型 （岭回归、lasso、KNN、SVM、神经网络）， 或者换成 PCA、特征筛选、填补缺失值——**泄漏立刻会咬人**， 就像上面这个例子一样。**规则要一刀切地遵守，不要赌。**

---

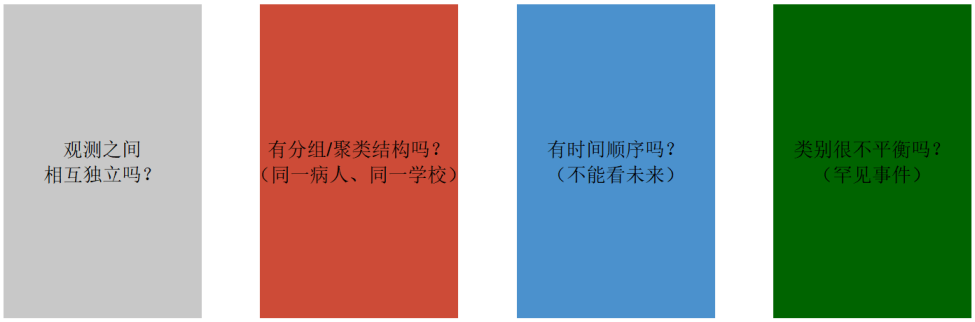
10 第二部分：适用情况

11 4. 什么情况下该用哪种 CV

12 4.1 决策树：先问数据是什么结构

不要上来就 `K=10` 。先回答三个问题：

选 CV 方案前先问这四个问题



对应的方案：

数据结构	该用的 CV	为什么
独立同分布（i.i.d.）	**K 折（K=5 或 10）**	标准情形
i.i.d. + 想要稳定估计	**重复 K 折**	压住划分带来的方差
有分组 / 聚类	**留一组法（Grouped / LOGO）**	同一组的观测不能跨越折
有层级（国家级→个人）	**按最高层分组**	预测目标是"新国家"
有时间顺序	**滚动 / 前推（rolling, forward-chaining）**	绝不能用未来预测过去
类别极不平衡	**分层 K 折（Stratified）**	保证每折的正例比例一致
空间数据	**空间分块（spatial block）**	相邻点高度相关
要用 CV 选模型**并**报告性能	**嵌套 CV**	见 §4.2

13 4.2 嵌套 CV：当"选模型"本身也要被评估

13.1 4.2.1 问题：用 CV 选模型，再用同一个 CV 报性能，会乐观

这是**最容易被忽略、也最影响结论**的一个坑。

错误流程（naive）：

1. 在全部数据上做 10 折 CV；
2. 挑 CV 误差最小的那个模型；
3. **就报这个最小的 CV 误差当作"模型的性能"。**

错在哪？ 第 3 步报的数字是 **12 个候选模型里的最小值。** 最小值天然偏小——你从 12 个里挑了最好的那个， 等于**用数据做了一次选择，却没有为这次选择付出代价。**

正确流程（nested）：

- **外层（outer）CV**：只用来**评估**，衡量"整套选模型流程"的性能；
- **内层（inner）CV**：只在**外层训练集内部**跑，用来**选模型。**

```
外层第 k 折：
  ├─ 外层训练集（9/10 的数据）
  │     └─ 内层 10 折 CV  →  选出最好的模型  ──┐
  └─ 外层测试集（1/10 的数据）                  │
        └─ 用上面选出的模型 → 算误差  ←─────────┘
```

13.2 4.2.2 实测：乐观了多少

```
sizes <- 1:12
forms <- lapply(sizes, function(s) reformulate(colnames(X)[1:s], response = "y"))

# 错误做法：在全部数据上选，然后报那个最小值
cv_naive <- sapply(forms, kfold_mse, data = dat, K = 10, seed = 303)
best_naive <- which.min(cv_naive)

c(selected_vars      = sizes[best_naive],
  reported_CV_error  = cv_naive[best_naive])
#>     selected_vars reported_CV_error
#>           4.00000           3.72508
```

```
# 正确做法：嵌套 CV
nested_cv <- function(data, forms, K_out = 10, K_in = 10, seed = 303) {
  set.seed(seed)
  n    <- nrow(data)
  out  <- sample(rep(1:K_out, length.out = n))
  sapply(1:K_out, function(k) {
    tr <- data[out != k, ]; te <- data[out == k, ]
    inner <- sapply(forms, kfold_mse, data = tr, K = K_in, seed = 500 + k)
    f_best <- forms[[which.min(inner)]]           # 选模型只在训练集里做
    mean((te$y - predict(lm(f_best, data = tr), newdata = te))^2)
  })
}

outer_errs <- nested_cv(dat, forms)
c(nested_CV_estimate = mean(outer_errs),
  outer_sd           = sd(outer_errs))
#> nested_CV_estimate           outer_sd
#>            3.90610            2.34052
```

把三个数字并排放：

```
# 另外算一个"真值"下界：真模型在整个总体上的期望测试误差
c(naive_select_and_report = cv_naive[best_naive],
  nested_CV_honest        = mean(outer_errs),
  optimism                = mean(outer_errs) - cv_naive[best_naive],
  optimism_pct            = 100 * (mean(outer_errs) - cv_naive[best_naive]) / cv_
naive[best_naive])
#> naive_select_and_report        nested_CV_honest                optimism
optimism_pct
#>                3.725079               3.906095               0.181016
4.859387
```

> 💡 **读这三个数：**
>
> - **naive 报出 3.725**；
> - **nested 报出 3.906**；
> - **乐观了 0.181 （约 5%）。**
>
> **在外层只有 10 折的情况下，这个差距看起来不大——但它是真实存在的系统性偏差**，而且会随着**候选模型数量**增加而放大。如果你在 5000 个变量里做筛选，naive 的乐观程度可以到**几十个百分点。**

> 🔴 **什么时候必须用嵌套 CV?**
>
> **只要你的最终模型是"用数据选出来的"，就应该用。**包括：- stepwise / 全子集变量选择；- 调超参数（岭回归的 λ、lasso 的 λ、树的深度……）；- 从多个模型族里挑一个（线性 vs 树 vs 神经网络）；- 有"先做特征工程再建模"的完整 pipeline。
>
> **什么时候不需要?** 如果你**事先就固定了一个模型**（比如"就用这 4 个变量"），没有任何"看着数据做的决定"，那普通 K 折 CV 就够了。

---

14 4.3 分组 / 聚类数据：留一组法

典型场景：

- 每个病人测了 4 次 → **同一病人的观测高度相关**；
- 每个学校抽了 30 个学生 → 同校学生共享学校效应；
- 每个受试者做了 3 个任务 → 同一受试者内部相关。

这里的核心问题不是"哪种 CV 更准"，而是"你到底要预测谁"。

14.1 4.3.1 一个会推翻直觉的例子

```
set.seed(42)
G <- 60; per <- 4
grp <- factor(rep(1:G, each = per))
u   <- rnorm(G, sd = 3)[as.integer(grp)]          # 个体效应，方差 = 9
x   <- rnorm(G * per)
yv  <- 1 + 2 * x + u + rnorm(G * per, sd = 0.3)  # 残差方差 = 0.09
gd  <- data.frame(y = yv, x = x, grp = grp)

c(var_u = 9, var_e = 0.09, ICC = 9 / (9 + 0.09))
#>    var_u    var_e      ICC
#> 9.000000 0.090000 0.990099
```

注意 ICC（组内相关系数）≈ 0.99 —— 也就是说**这个数据的 99% 变异来自"个体之间"，几乎不来自个体内部。**

我们用 $y \sim x$ 这一个模型，跑两种 CV：

```
# (a) 普通随机 10 折（无视分组）——重复 50 次取平均
set.seed(1)
random_cv <- mean(sapply(1:50, function(s) {
  set.seed(s); fd <- sample(rep(1:10, length.out = nrow(gd)))
  mean(sapply(1:10, function(k) {
    tr <- gd[fd != k, ]; te <- gd[fd == k, ]
    mean((te$y - predict(lm(y ~ x, data = tr), newdata = te))^2)
  }))
}))

# (b) 留一人法：整人一起留出
logo_cv <- mean(sapply(levels(gd$grp), function(g) {
  tr <- gd[gd$grp != g, ]; te <- gd[gd$grp == g, ]
  mean((te$y - predict(lm(y ~ x, data = tr), newdata = te))^2)
}))

c(random_10fold = random_cv, leave_one_person_out = logo_cv)
#>        random_10fold leave_one_person_out
#>              12.0010              12.2401
```

两种 CV 几乎一样：12 vs 12.24。 看起来"分组不分组的，没什么差别嘛"。

现在算两个理论真值。 因为数据是我们自己造的，我们**知道**答案：

```r
m_pool <- lm(y ~ x, data = gd)
V <- vcov(m_pool)                          # 系数的不确定度

# 真值 1：预测【已经见过的那个人】的一次新观测
#   个体效应 u 可以从他 4 条训练数据里估出来，误差只剩 var(e) + 估计误差
truth_seen <- 0.09 + 0.09 / 4 + V[1, 1] + V[2, 2]

# 真值 2：预测【一个全新的人】
#   新人的 u* ~ N(0, 9) 完全未知，方差 9 全部计入
truth_new  <- 9 + 0.09 + V[1, 1] + V[2, 2]

c(truth_new_obs_of_seen_person = truth_seen,
  truth_brand_new_person       = truth_new)
#> truth_new_obs_of_seen_person       truth_brand_new_person
#>                      0.21781                      9.19531
```

把四个数字并排放：

```r
data.frame(
  what = c("随机 10 折 CV", "留一人法 CV",
           "真值：见过的人的新观测", "真值：一个全新的人"),
  value = round(c(random_cv, logo_cv, truth_seen, truth_new), 3)
) |> transform(
  ratio_to_CV = round(c(1, 1, truth_seen / random_cv, truth_new / random_cv), 2)
)
#>                      what  value ratio_to_CV
#> 1           随机 10 折 CV 12.001        1.00
#> 2             留一人法 CV 12.240        1.00
#> 3 真值：见过的人的新观测  0.218        0.02
#> 4     真值：一个全新的人  9.195        0.77
```

🔴 **这张表是本节的全部要点，而且结论和你想的可能相反：**

	数值	相对 CV 估计
随机 10 折 CV	12	1.0×
留一人法 CV	12.24	1.0×
真值：见过的人的新观测	**0.22**	**0.02×**
真值：一个全新的人	**9.2**	0.77×

两种 CV 都没能给你正确答案，而且它们的差别小到可以忽略。

- 对"**预测已见过的人**"这件事，CV（12） **高估了 55 倍**；
- 对"**预测一个全新的人**"这件事，CV（12） 又**恰好接近**真值（9.2）。

为什么两种 CV 都不对？ 因为**问题不在 CV 方案，在模型本身**：

> $y \sim x$ 这个模型**根本没有"个体"这个概念。** 它对所有人给出同样的截距。 而数据里 99% 的变异是**人与人之间的差异—— 这个模型在结构上就无法表达这个差异。**
>
> 所以 CV 算出来的 12， 是"这个**错误模型**在留出数据上的误差"， 它既不是"见过的人"的误差（那个是 0.22）， 也不是"新人"的误差（9.2）——**它俩差 42 倍。**

14.2 4.3.2 那什么时候"留一组"才是关键？

上面的例子说明：**CV 方案选错，不一定是最大的问题；模型选错更致命。** 但"留一组"在下面这些情形里**确实是决定性的**：

情形	随机 K 折会怎样	为什么必须留一组
组内观测几乎重复（同一人多次测量、同一图像的多个补丁）	训练集里有这个人的其他测量，等于**见过答案**	测试折必须是不认识的人
特征里含"组身份"信息（如"病人历史均值"这种 target encoding）	模型可以**直接背出**这个人的水平	留一组后这个特征对新组无意义
目标是全新组（新医院、新学校、新设备）	随机 CV 回答的是另一个问题	只有留一组才对应部署场景
组数少但组很大	训练/测试共享组，高估泛化能力	留一组才能暴露"跨组"的风险

而 §3.5 那个特征筛选的例子，就是第二行的极端版本—— 只不过那里泄漏的不是"组身份"，而是"全样本的相关系数"。

> 💡 **所以正确的思考顺序是：**
>
> 1. **先问：我要预测的对象是什么？** （新人？新组？未来的时间点？）
> 2. **再问：我的模型能表达这个结构吗？** （有没有个体效应项？要不要混合模型？）
> 3. **最后才问：CV 该怎么分折，才能模拟第 1 步的场景？**
>
> **反过来做（先选 CV 再想问题）是最常见的错误。**

14.3 4.3.3 一个诊断工具：ICC

组内相关系数（intraclass correlation, ICC） 告诉你"分组到底有多重要"：

$$\text{ICC} = \frac{\sigma_u^2}{\sigma_u^2 + \sigma_e^2}$$

- 本例 0.99 —— **极高**，必须认真处理分组；
- ICC 接近 0 ⇒ 组之间没差别，随机 K 折基本没问题；
- **实践中 ICC > 0.1 就应该考虑分组结构了。**

怎么估 ICC？ 最简单的方法是拟合一个"随机截距模型"：

```
# 用组均值近似：组间方差 / 总方差
gm  <- tapply(gd$y, gd$grp, mean)
icc_hat <- var(gm) / var(gd$y)
c(ICC_rough_estimate = round(icc_hat, 3), n_groups = length(gm))
#> ICC_rough_estimate           n_groups
#>               0.81              60.00
```

> ⚠️ **但这个粗略估计是有偏的**（组均值本身的抽样误差也算进去了）。 正规做法是用混合模型（`lme4::lmer(y ~ x + (1 | grp))`） 或矩估计法把 σ_e^2 和 σ_u^2 分开估。 这超出本课范围，但"**先算个 ICC 看看要不要处理分组**"**是个好习惯。**

---

15 4.4 时间序列：不能用随机 K 折

核心禁忌：绝对不能用未来数据预测过去。

如果随机分折，训练集里会包含"未来"的观测， 于是模型**偷看了未来**，测试误差虚低得离谱。在金融、需求预测、流行病学里，这会造成灾难性的错误结论。

正确做法： 折必须按时间**连续分块**，并且训练集永远在测试集**之前。**

```
随机 K 折（❌）          滚动 / 前推（✅）
时间 →                   时间 →
[■□■|□■|■■|□■]          [■■■|□          ]  第1折
 ■=训练 □=测试           [■■■■|□         ]  第2折
                          [■■■■■|□        ]  第3折
                          训练集只增不减，测试集永远在后面
```

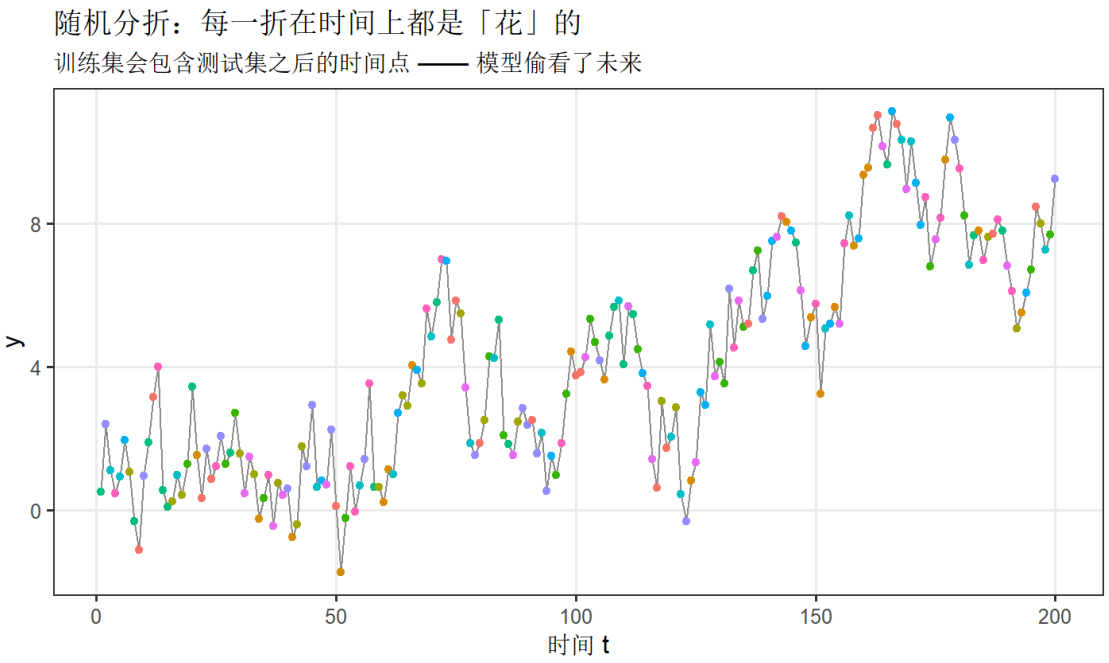
```r
# 造一个有强自相关的序列
set.seed(9)
Tt <- 200
tt <- 1:Tt
yt <- 0.05 * tt + as.numeric(arima.sim(list(ar = 0.9), n = Tt)) + rnorm(Tt, sd =
0.5)
tsd <- data.frame(t = tt, y = yt)

# (a) 随机 K 折（错误）
set.seed(2); foldr <- sample(rep(1:10, length.out = Tt))
rand_ts <- mean(sapply(1:10, function(k) {
  tr <- tsd[foldr != k, ]; te <- tsd[foldr == k, ]
  mean((te$y - predict(lm(y ~ t, data = tr), newdata = te))^2)
}))

# (b) 滚动 / 前推（正确）
roll_ts <- mean(sapply(1:9, function(k) {
  cut <- floor(Tt * (0.5 + 0.05 * (k - 1)))
  tr  <- tsd[1:cut, ]
  te  <- tsd[(cut + 1):min(cut + 20, Tt), ]
  mean((te$y - predict(lm(y ~ t, data = tr), newdata = te))^2)
}))

c(random_kfold_WRONG = rand_ts,
  rolling_forward_OK = roll_ts)
#> random_kfold_WRONG rolling_forward_OK
#>            3.42279            5.85749
```

```r
tsd$fold_rand <- factor(foldr)
ggplot(tsd, aes(t, y)) +
  geom_line(colour = "grey55", linewidth = 0.4) +
  geom_point(aes(colour = fold_rand), size = 1.4, show.legend = FALSE) +
  labs(x = "时间 t", y = "y",
       title = "随机分折：每一折在时间上都是「花」的",
       subtitle = "训练集会包含测试集之后的时间点 —— 模型偷看了未来") +
  theme_bw(base_size = 12) + theme(panel.grid.minor = element_blank())
```



🔴 **判断标准：“我预测的时刻，在训练数据之后吗？”** 如果不是，这个 CV 结果**没有意义**，不管它看起来多好。

补充： 时间序列还有更细的方案——**滑动窗口（sliding window）** 固定训练长度向前滚，**扩展窗口（expanding window）**训练集只增不减。 选哪个取决于”预测时我能拿到多少历史数据”。

---

16 4.5 分层 K 折：不平衡数据

问题： 如果正例只占 2%，纯随机分折可能某一折里**一个正例都没有**， 那一折的误差就无法定义（或者极不稳定）。

做法： 在**每个类别内部**分别做随机划分，保证每折的类别比例与总体一致。

```r
set.seed(5)
n3 <- 300
cls <- rbinom(n3, 1, prob = 0.05)            # 5% 正例
table(cls)
#> cls
#>   0   1
#> 286  14

# (a) 纯随机分折
set.seed(6)
fold_random <- sample(rep(1:10, length.out = n3))

# (b) 分层分折：在每一类内部各自分折
set.seed(6)
fold_strat <- integer(n3)
for (cl in c(0, 1)) {
  idx <- which(cls == cl)
  fold_strat[idx] <- sample(rep(1:10, length.out = length(idx)))
}

data.frame(
  fold              = 1:10,
  positives_random  = as.integer(table(factor(fold_random[cls == 1], levels = 1:1
0))),
  positives_strat   = as.integer(table(factor(fold_strat[cls == 1],  levels = 1:1
0)))
)
#>    fold positives_random positives_strat
#> 1     1                0               2
#> 2     2                1               2
#> 3     3                3               2
#> 4     4                2               2
#> 5     5                0               1
#> 6     6                3               1
#> 7     7                2               1
#> 8     8                0               1
#> 9     9                1               1
#> 10   10                2               1
```

> **注意看 `random` 那一列：** 有些折里正例是 0 或 1。 如果损失是"错误率"或 AUC，这一折的估计会极其不稳。 **分层之后每一折的正例数几乎一样。**
>
> **经验：** 分类问题**默认就应该分层**，代价几乎为零， 在类别不平衡时收益很大。

17 4.6 CV 做不到什么

必须知道 CV 的边界，否则会过度自信。

CV 做不到	为什么	该用什么
给出无偏的性能估计（流程含选择时）	选择本身消耗了信息	嵌套 CV
告诉你模型对不对	它只衡量**预测**，不衡量**因果**、不检查函数形式	残差诊断、领域知识
替代外部验证	CV 是**同一批数据内部**的重采样	真正的外部数据 / 时间外样本
自动选出可解释的模型	预测最优 ≠ 解释最优	领域知识 + 简约性
处理分布漂移（distribution shift）	它假设训练和未来的数据同分布	领域适应、重新加权
在极小样本下可靠	一切估计都极不稳	见 §6.3
用来做后续统计推断	选完模型再算 p 值，p 值有偏	样本分裂（sample splitting）/ 选择性推断

> 🔴 **最容易被忘掉的一条：CV 不能替代外部验证。** 如果你的模型要部署到**另一个医院、另一个国家、另一年**的数据上，**只有真正的外部数据能验证它。** CV 再好，也只是"在这批数据内部看起来好"。

---

18 第三部分：动手做

19 5. 完整案例：seatpos 上从头实现 CV

数据： `faraway::seatpos`，38 位驾驶员的身体尺寸与他们把座位调到的位置 `hipcenter`。**目标：找出一个预测性能好的模型。**

19.1 5.1 四个候选模型

```
sp <- seatpos
fms <- list(
  "Ht"              = hipcenter ~ Ht,
  "Ht + Leg"        = hipcenter ~ Ht + Leg,
  "Age + HtShoes + Leg" = hipcenter ~ Age + HtShoes + Leg,
  "full (8 vars)"   = hipcenter ~ Age + Weight + HtShoes + Ht +
                                  Seated + Arm + Thigh + Leg
)
```

> ⚠️ **这里有个真实的坑：** 不要用 `reformulate()` 去拼公式！ `seatpos` 里有一列就叫 `Leg`，而 `reformulate()` 默认把响应变量命名为 `"y"` —— 一旦你的数据里恰好也有 `y` 列，公式就会引用错的变量。 **直接把公式写全**，多打几个字，少一个 bug。

19.2 5.2 四个准则对比

```
cv_kfold <- function(formula, data, K = 10, seed = 7) {
  set.seed(seed)
  n    <- nrow(data)
  fold <- sample(rep(1:K, length.out = n))
  mean(sapply(1:K, function(k) {
    tr <- data[fold != k, ]; te <- data[fold == k, ]
    mean((te$hipcenter - predict(lm(formula, data = tr), newdata = te))^2)
  }))
}
cv_repeated <- function(formula, data, K = 10, R = 50) {
  v <- sapply(1:R, function(r) cv_kfold(formula, data, K = K, seed = 5000 + r))
  c(mean = mean(v), sd = sd(v))
}

case_tab <- do.call(rbind, lapply(names(fms), function(nm) {
  fm <- fms[[nm]]
  m  <- lm(fm, data = sp); e <- resid(m); h <- hatvalues(m)
  rr <- cv_repeated(fm, sp)
  data.frame(
    model      = nm,
    k_params   = length(coef(m)),
    train_MSE  = mean(e^2),
    adj_R2     = summary(m)$adj.r.squared,
    LOOCV      = mean((e / (1 - h))^2),
    CV10_mean  = rr["mean"],
    CV10_sd    = rr["sd"]
  )
}))
case_tab |> mutate(across(where(is.numeric), \(v) round(v, 4)))
#>                     model k_params train_MSE adj_R2   LOOCV CV10_mean  CV10_sd
#> mean                   Ht        2   1253.05 0.6282 1387.64   1392.87  56.2990
#> mean1            Ht + Leg        3   1179.86 0.6399 1408.70   1413.16  72.2751
#> mean2 Age + HtShoes + Leg        4   1104.15 0.6531 1412.61   1418.69  78.2255
#> mean3       full (8 vars)        9   1085.84 0.6001 1975.41   2009.77 193.0825
```

19.3 5.3 怎么读这张表

四个准则给出了不同的"最优"：

准则	选出的模型	合理性
训练误差	full (8 vars)	❌ 必然选最复杂的
Adjusted R^2	Age + HtShoes + Leg	⚠️ 倾向于较复杂的
LOOCV	**Ht**	✅
重复 10 折 CV	**Ht**	✅

> 💡 **这张表最有价值的观察：**
>
> **看 `CV10_mean ± CV10_sd` 这两列。** `Ht` 是 1392.9 ± 56.3，`Age + HtShoes + Leg` 是 1418.7 ± 78.2。**两者的一倍标准差范围大幅重叠。**
>
> 而 Adjusted R^2 说 `Age + HtShoes + Leg`（0.653）明显好于 `Ht`（0.628）。
>
> **哪一个对？——CV 更接近你真正关心的问题**（预测新驾驶员），而 Adjusted R^2 是一个**样本内的**、带固定惩罚项的近似。在 $n = 38$ 这么小的样本上，CV 偏好**更简单**的模型是合理且常见的现象。
>
> **但重点不是"Ht 赢了"这个结论本身，而是：结论依赖于你用什么准则，而准则之间会打架。你必须说清楚你用的是什么，以及为什么。**

19.4 5.4 选不出来的时候怎么办：一倍标准差规则

```
best   <- which.min(case_tab$CV10_mean)
cutoff <- case_tab$CV10_mean[best] + case_tab$CV10_sd[best]

data.frame(
  model        = case_tab$model,
  CV10_mean    = round(case_tab$CV10_mean, 1),
  within_1se   = case_tab$CV10_mean <= cutoff,
  k_params     = case_tab$k_params
) |> transform(cutoff = round(cutoff, 1))
#>                 model CV10_mean within_1se k_params cutoff
#> 1                  Ht    1392.9       TRUE        2 1449.2
#> 2           Ht + Leg    1413.2       TRUE        3 1449.2
#> 3 Age + HtShoes + Leg    1418.7       TRUE        4 1449.2
#> 4       full (8 vars)    2009.8      FALSE        9 1449.2

c(chosen_by_one_SE_rule = case_tab$model[case_tab$CV10_mean <= cutoff][
    which.min(case_tab$k_params[case_tab$CV10_mean <= cutoff])])
#> chosen_by_one_SE_rule
#>                  "Ht"
```

一倍标准差规则（one-standard-error rule）：

> 在所有"CV 值 ≤ 最优值 + 1 个标准差"的模型里，**选最简单的那个。**

理由： 既然 CV 区分不出这些模型，那就**按简约原则**选。 这既降低了过拟合风险，也让模型更好解释、更好维护。

---

20 6. 常见陷阱与最佳实践

20.1 6.1 十个最常见的错误

#	错误	后果	正确做法
1	**预处理放在折外**	性能虚高，可能完全无效	标准化、填补、PCA、筛选**全部折内做**
2	**用 CV 选模型又报同一个 CV**	系统性乐观（§4.2）	嵌套 CV
3	**有分组却用随机 K 折**	回答了错误的问题；若组内近乎重复则会严重低估（§4.3）	留一组法，并检查模型能否表达组结构
4	**时间序列用随机 K 折**	偷看未来，结论无意义	滚动 / 前推
5	**只跑一次 CV 就下结论**	结论可能是噪声	重复 K 折，报告 ± sd
6	**用训练误差选模型**	永远选最复杂的	用 CV 或 AIC/BIC
7	**分类不平衡不分层**	某些折没有正例	分层 K 折
8	**LOOCV 用在非线性模型上**	帽子矩阵公式不成立	老实重拟合，或用 K 折
9	**报了 CV 却不说怎么分的**	结果不可复现、无法评估	写明 K、种子、分组变量
10	**把 CV 当成外部验证**	部署后性能崩塌	真正的外部数据集

20.2 6.2 一份"可以交出去"的 CV 报告清单

写作业或论文时，**逐条检查**：

- ☐ **损失函数是什么？（MSE？MAE？错误率？）为什么选它？**
- ☐ **K 是多少？为什么？**

- ☐ **随机种子是什么？（可复现）**
- ☐ **重复了多少次？ 报告的是均值还是单次？**
- ☐ **不确定性报了吗？（标准差 / 范围）**
- ☐ **所有预处理都确认在折内做了吗？**
- ☐ **分折方式和数据结构匹配吗？（独立 / 分组 / 时间 / 分层）**
- ☐ **如果是"选出来的模型"，用了嵌套 CV 吗？**
- ☐ **有没有和更简单的基线比？（比如只用截距的模型）**
- ☐ **结论有没有说清楚适用范围？（预测什么样的对象？）**

20.3 6.3 小样本怎么办

n 很小时（比如 `seatpos` 的 38，或更小），CV 会遇到两个问题：

1. **训练集太小** ⇒ 偏差大（模型学不好）；
2. **折与折之间的差异大** ⇒ 方差大，结论不稳。

可用的对策：

对策	说明
用 LOOCV	训练集最大（$n-1$），偏差最小。计算量对线性模型不是问题
重复很多次 K 折	压方差，还能看分布
报告不确定性	一定要报 sd 或区间，不能只报一个数
一倍标准差规则	选不出就选最简单的
不要过度比较模型	在 $n = 38$ 上比较 4 个模型，差距基本都在噪声里
考虑 AIC/BIC	它们用全部数据拟合，在小样本下有时比 CV 更稳
考虑 C_p、GCV	也是基于全部数据的准则，见变量选择那一节

> ⚠️ **一句诚实话： 在 $n = 38$ 的数据上，任何模型选择方法的结果都应该被当作"建议"而不是"结论"。** CV 能帮你排除明显糟糕的模型，但它**没有办法**在小样本上精确区分好模型和次好模型。 **诚实报告"这几个模型我们区分不出来"，比硬选一个更专业。**

21 附录

21.1 A. 一张速查表

场景	方法	R 里怎么做
标准 i.i.d. 回归/分类	10 折 CV	本讲义 §1.2 的 `cv_error()`
想要更稳的估计	重复 10 折	循环 + 不同种子
线性模型 + 平方损失	LOOCV 闭式	`mean((resid(m)/(1-hatvalues(m)))^2)`
样本极小	LOOCV 或重复 K 折	同上
有分组	留一组法	`sapply(levels(d$grp), ...)`
时间序列	滚动 / 前推	按时间切块，训练集只取前面
分类不平衡	分层 K 折	各类内分别分折
要选模型并报性能	嵌套 CV	外层评估、内层选择

21.2 B. 与其它准则的关系

准则	用全部数据拟合?	用新数据评估?	适用
训练误差	✅	❌	**永远不要用来选模型**
Adjusted R^2	✅	❌（只是惩罚项近似）	快速比较、同族模型
AIC / BIC	✅	❌（渐近近似）	大样本、嵌套模型比较
C_p	✅	❌	线性模型
CV	❌（要重拟合）	✅ **真的留出**	**通用，任何模型任何损失**
外部验证	—	✅ ✅	最终的金标准

CV 相对 AIC/BIC 的三个优势：

1. **不依赖渐近理论**——直接做留出，小样本也说得通；
2. **能做任何模型**——AIC 要算似然和有效参数个数，随机森林/GNN 很难；
3. **能配合任何损失**——MAE、AUC、分位数损失都可以。

AIC/BIC 相对 CV 的优势：

1. **只拟合一次**，快得多；
2. **对小样本的方差更小**（因为用了全部数据拟合）；

3. 有明确的理论性质（如 AIC 的渐近最优性）。

> **实践建议：** 两者都跑，看结论是否一致。 **一致 → 更有信心；打架 → 说明数据里的信号不足以区分模型，要谨慎下结论。**

21.3 C. 本文用到的 R 技能

技能	用在哪
手写 K 折（`sample` + `rep` + 循环）	§1.2
`hatvalues()` 实现 LOOCV 闭式	§3.3
用 `replicate` / `sapply` 做重复实验	§2、§3.4
`reformulate()` 动态构造公式（**及它的坑**）	§5.1
分层抽样（按类别分别分折）	§4.5
按组留出	§4.3
时间序列切块	§4.4
嵌套循环实现嵌套 CV	§4.2

21.4 D. 延伸阅读

- James, Witten, Hastie, Tibshirani. *An Introduction to Statistical Learning*, Chapter 5（交叉验证与 bootstrap）——**最适合本课的下一本读物。**
- Hastie, Tibshirani, Friedman. *The Elements of Statistical Learning*, Chapter 7（模型评估与选择）——更理论化。
- Arlot, Celisse (2010). *A survey of cross-validation procedures for model selection*. Statistics Surveys.——想深入 CV 理论看这篇。
- Varma, Simon (2006). *Bias in error estimation when using cross-validation for model selection*. BMC Bioinformatics.——嵌套 CV 的经典参考文献。

```
sessionInfo()
#> R version 4.4.2 (2024-10-31 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 26200)
#>
#> Matrix products: default
#>
#>
#> locale:
#> [1] LC_COLLATE=Chinese (Simplified)_China.utf8  LC_CTYPE=Chinese (Simplified)_
China.utf8
#> [3] LC_MONETARY=Chinese (Simplified)_China.utf8 LC_NUMERIC=C
#> [5] LC_TIME=Chinese (Simplified)_China.utf8
#>
#> time zone: Asia/Shanghai
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] faraway_1.0.9   patchwork_1.3.0 scales_1.4.0    tidyr_1.3.1     dplyr_1.1.
4     ggplot2_4.0.3
#>
#> loaded via a namespace (and not attached):
#>  [1] Matrix_1.7-1       gtable_0.3.6       jsonlite_1.8.9     compiler_4.4.2
Rcpp_1.1.1
#>  [6] tidyselect_1.2.1   jquerylib_0.1.4    splines_4.4.2      boot_1.3-31
yaml_2.3.10
#> [11] fastmap_1.2.0      lattice_0.22-6     R6_2.5.1           labeling_0.4.3
generics_0.1.3
#> [16] knitr_1.49         MASS_7.3-61        tibble_3.2.1       nloptr_2.1.1
minqa_1.2.8
#> [21] bslib_0.8.0        pillar_1.9.0       RColorBrewer_1.1-3 rlang_1.2.0
utf8_1.2.4
#> [26] cachem_1.1.0       xfun_0.49          sass_0.4.9         S7_0.2.2
cli_3.6.3
#> [31] withr_3.0.2        magrittr_2.0.3     digest_0.6.37      grid_4.4.2
lme4_1.1-35.5
#> [36] nlme_3.1-166       lifecycle_1.0.5    vctrs_0.7.3        evaluate_1.0.1
glue_1.8.0
#> [41] farver_2.1.2       fansi_1.0.6        rmarkdown_2.29     purrr_1.2.2
tools_4.4.2
#> [46] pkgconfig_2.0.3    htmltools_0.5.8.1
```